UNIT 3

➢ **What is Grammar**

Grammar in automata theory is a standard way of representing the language. The grammar contains a set of production rules that make up the language strings.

The set of all possible strings that can be derived from grammar is known as the language of that grammar. The symbol "G" mostly denotes grammar .

A Grammar is a 4-tuple such that-

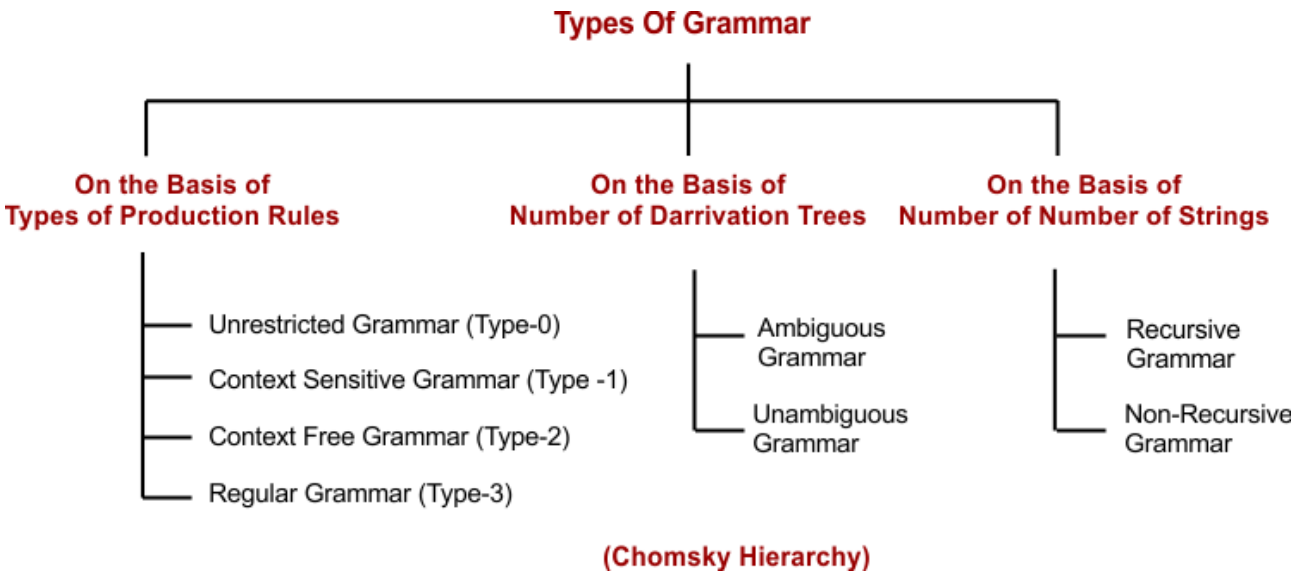
G = (V , T , P , S)

where-

- V = Finite non-empty set of non-terminal symbols
- T = Finite set of terminal symbols
- P = Finite non-empty set of production rules
- S = Start symbol

Types of Grammars

There are three significant categories of grammar, as explained in the following diagram



1.Regular Grammar (RL)

Regular Grammar generates the Regular Languages, and the Finite Automata Machine accepts all regular languages. It is also called Type-3 grammar.



Regular languages are of two types.

I. Finite Regular Languages
II. Infinite Regular Languages

Finite Automata Machine is also of two types

I. Deterministic Finite Automata (DFA)
II. Non-Deterministic Finite Automata (NDFA)

All regular languages, either finite or infinite, are accepted by NDFA. However, all regular languages and some infinite languages are accepted by DFA.

2. Context Free Grammar

Context-free Grammar generates Context-free Languages, and the Pushdown Automata Machine accepts all context-free languages. Context-free Grammar is also called Type-2 grammar.



3. Context Sensitive Grammar (CSG)

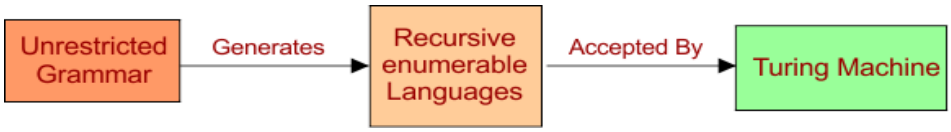
Context-sensitive Grammar generates context-sensitive languages, and linear bound automata machines accept all context-sensitive languages. Context-free Grammar is also called Type-1 grammar.



Note: Context-sensitive languages and their machines are not deterministic or non-deterministic.

4. Unrestricted Grammar

Unrestricted Grammar generates the Recursive enumerable Languages, and the Turing Machine accepts all Recursive enumerable languages. It is also called Type-0 grammar.



➢ **What is Context Free Grammar?**

Context Free Grammar (CFG) is a type of formal grammar that is used to generate every conceivable string in a given formal language. It's like a recipe that tells you how to create different strings in a language

*Context Free Grammar provides no mechanism to restrict the usage of the production rule $A \rightarrow \alpha$ within some specific context unlike other types of grammars. That is why it is called as **"Context Free"** Grammar.*

A context free grammar G is defined by four tuples: $G = (V, T, P, S)$

Here's what they mean:

- G refers to a grammar that is made up of sets of different production rules. These rules are used to generate the strings of a language.
- T represents the final set of terminal symbols, which are usually denoted by lower case letters. Terminal symbols never appear on the left-hand side of the production rule and are always on the right-hand side.
- V represents the final set of non-terminal symbols, which are usually denoted by capital letters.
- P is a finite set of production rules. These rules are used to replace the non-terminal symbols (on the left side of a production) with other terminals (on the right side of a production).
- Production rules have the following form:

  Non-terminal → (terminals + non-terminals) *

P is a finite set of productions of the form A →α , where A is a variable and **α∈ (V+T)***

- S is the start symbol. This symbol is used to derive the string.

There is a finite set of productions or rules that represent the recursive definition of a language. Each production consists of :

- A variable that is being defined by the production. This is called head of production.
- The production symbol →
- A string of Zero or more terminals and Variables. This string is called the body of the production, represents one way to form the string in the language of the head.

The string in a CFG is derived using the start symbol. This string can be derived by continuously replacing a nonterminal with the right-hand side of a production, until all the non-terminals are replaced by terminal symbols.

In Context-Free Grammars (CFG), a one-step derivation means applying a single production rule to replace a non-terminal symbol with its right-hand side, while a zero or more step derivation allows for zero or more applications of production rules.

➢ Language of Grammar (Context Free Grammar):

• A language that is defined by some CFG is called a context-free language.

• There are CFL's that are not regular language

Regular Languages ⊂ Context-Free Languages

All regular languages are CFLs ✓ But not all CFLs are regular ✗

• Intuition: CFL's can count two things, not three.

Let G = (V, T, P and S) is a context free grammar. Then the language of G denoted by L(G) is the set of terminal strings that have derivation from the start symbol in G. i.e.

L (G) = {x ε T* | S→* x} The language generated by a CFG is called the Context Free Language (CFL).

➢ Regular Grammar

A grammar is called **regular** if all its productions are either left-linear or right-linear, with at most one non-terminal on the right-hand side.

Valid Right-Linear Grammar non-terminal= Terminal + non-terminal

Ex : S→aS|bS|a

Valid Left-Linear Grammar non-terminal= non Terminal + terminal

Ex : S→Sa|Sb|a

INVALID regular grammar (Mixed Grammar) $S \rightarrow aS \mid Sb \mid a$

➢ Applications of Context Free Grammar (CFG)

- Context Free Grammars are used in Compilers (like GCC) for parsing. In this step, it takes a program (a set of strings).
- Context Free Grammars are used to define the High Level Structure of a Programming Language.
- Every Context Free Grammar can be converted to a Parser which is a component of a Compiler that identifies the structure of a Program and converts the Program into a Tree.
- Document Type Definition in XML is a Context Free Grammar which describes the HTML tags and the rules to use the tags in a nested fashion.

➢ **Classification of Context Free Grammars**

CFG is classified on the basis of the following two properties:

1) The Number of Generated Strings

- CFG is *non-recursive* if it produces a finite number of strings, or the grammar becomes a non-recursive grammar.
- The grammar is recursive if CFG can produce an endless amount of strings, repeating grammar.

The parser creates a derivation tree or a parse tree out of the source code during compilation by using the language's grammar. There must be no ambiguity in the grammar. Parsing must not employ unclear grammar.

2) The Number of Derivation Trees

- If there is just one derivation tree, the CFG is clear/unambiguous.
- If there are multiple derivation trees, the CFG is unclear/ambiguous.

➢ **To create a string from a context-free grammar, follow these steps:**

- Begin the string with a start symbol.
- Apply one of the production rules to the start symbol on the left-hand side by replacing the start symbol with the right-hand side of the production.
- Repeat the process of selecting nonterminal symbols in the string, and replacing them with the right-hand side of some corresponding production, until all non-terminals have been replaced by terminal symbols. Note, it could be that not all production rules are used.

We say $\alpha A \beta => \alpha\gamma\beta$ if $A \rightarrow \gamma$ is a production.

Iterated Derivation $\Rightarrow*$ means zero or more derivation steps.

• Basis: $\alpha \Rightarrow* \alpha$ for any string α.

• Induction: if $\alpha \Rightarrow* \beta$ and $\beta \Rightarrow \gamma$, then $\alpha \Rightarrow* \gamma$.

➢ **CFG Derivation – Types of Derivations**

Derivation is generation of string from grammar.

Derivation is a process of generating a string by repeatedly applying production rules, starting from start symbol until only terminals remain.

We can use CFG derivation to find out

Which string is produced by grammar and Specific string is generated by G or not

Here we have two approaches to derive string from G

1. Leftmost Derivation $\underline{\Rightarrow lm}$:
   Here leftmost variable or non-terminal is replaced first.
2. Rightmost Derivation $\Rightarrow$rm :
   Here rightmost Variable / Non-terminal is replaced first

We do a *leftmost derivation* of a string from a grammar by

1. Begin with a string consisting only of the start symbol.
2. Pick *the leftmost* variable occurrence in the current string.
3. Pick any production that has that variable on the left of the →.
4. Replace the chosen occurrence of that variable by the right-hand side of the chosen production.
5. Repeat steps 2-4 until the string contains only terminals.

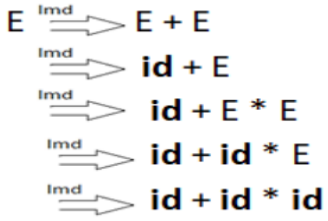
We do a *rightmost derivation* of a string from a grammar by

1. Begin with a string consisting only of the start symbol.
2. Pick *the rightmost* variable occurrence in the current string.
3. Pick any production that has that variable on the left of the →.
4. Replace the chosen occurrence of that variable by the right-hand side of the chosen production.
5. Repeat steps 2-4 until the string contains only terminals.

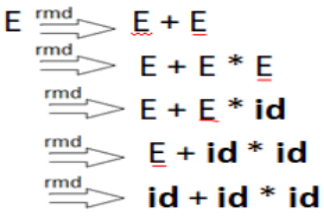
Consider the G,

E → E + E | E * E | (E) | - E | **id**

Derive the string **id + id * id** using leftmost derivation and rightmost derivation.



(a)



(b)

Sentential Forms

Derivations from the start symbol produce strings that have a special role. We call these "sentential forms". i.e. if G=(V,T,P,S) is a CFG, then any string α is in (V U T)* such that S→* α is a sentential form.

If S→lm* α, then α is a left sentential forms, and if S→rm * α, then α is a right sentential form.

Any string of variables and/or terminals derived from the start symbol is called a sentential form. Formally, α is a sentential form iff S ⇒∗ α.

If G is a CFG, then L(G), the language of G is {w | S ⇒∗ w}. Note: w must be a terminal string, S is the start symbol.

Example

Grammar: $S \rightarrow aSb \mid ab$

Derivation of aabb: $S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aabb$

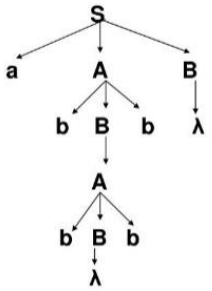
Sentential forms here are: S aSb aaSbb aabb

➢ **Derivation Tree / Parse Tree:**

Parse tree is a tree representation of strings of terminals using the productions defined by the grammar. A parse tree pictorially shows how the start symbol of a grammar derives a string in the language. Parse tree may be viewed as a graphical representation for a derivation that filters out the choice regarding the replacement order

Example:

- S -> aAB
  A -> bBb
  B -> A | λ
- Show derivation tree for derivation of abbbb

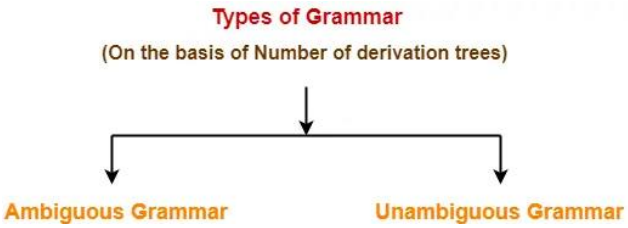


Parse trees are an alternative representation to derivations and recursive inferences
• Parse trees are trees labelled by symbols of a particular CFG.
• *Leaves*: labelled by a terminal or ε.
• *Interior nodes*: labelled by a variable. Children are labelled by the right side of a production for the parent.
• *Root*: must be labelled by the start symbol

Yield of the parse tree.

The concatenation of the labels of the leaves in left-to-right order is called the yield of the parse tree.
• For every parse tree, there is a unique leftmost and a unique rightmost derivation.
• We'll prove:
1 If there is a parse tree with root labelled A and yield w, then $A \Rightarrow^* lm\ w$.
2 If $A \Rightarrow^* lm\ w$, then there is a parse tree with root A and yield w

- **Ambiguity in Grammar:**



A Grammar G = (V, T, P and S) is said to be ambiguous if there is a string w ∈ L (G) for which we can derive two or more distinct derivation tree rooted at S and yielding w.

In other words, a grammar is ambiguous if it can produce more than one leftmost or more than one rightmost derivation for the same string in the language of the grammar.

If the grammar is not ambiguous then we call it **unambiguous** grammar.
No method can automatically detect and remove the ambiguity, but we can remove the ambiguity by re-writing the whole grammar without ambiguity.

Example: Given grammar G : E → E+E | E*E | (E) | - E | id

The sentence id+id*id has the following two distinct leftmost derivations:

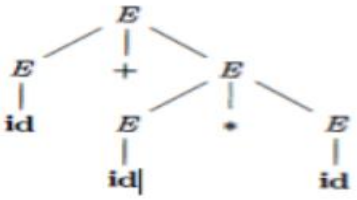
Leftmost Derivations -1

E → E+ E
E → id + E
E → id + E * E
E → id + id * E
E → id + id * id

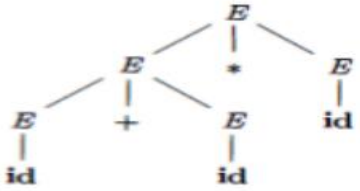
Leftmost Derivations -2

E → E* E
E → E + E * E
E → id + E * E
E → id + id * E
E → id + id * id

The two corresponding parse trees are :



(a)



(b)

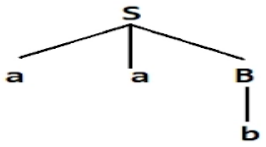
So the above grammar ambiguous

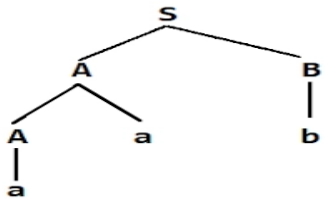
Example 2

Let production rule is given as:

```
S -> AB|aaB     A -> a|Aa  B -> b
```

Let us generate string **aab** from the given grammar. Parse trees for generating string **aab** are as follows :





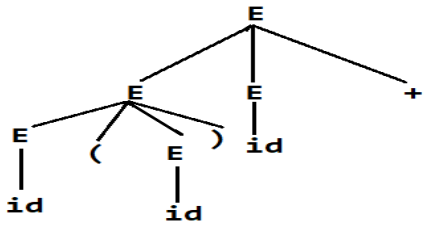
Here for the same string, we are getting more than one parse tree. Hence, grammar is ambiguous grammar.

Example 3

Let production rule is given as:

E -> EE+ E → E(E) | id

Parse tree for **id(id)id** + is:



Only one parse tree is possible for id(id)id+, so the given grammar is unambiguous.

Removal of Ambiguity from CFG

Conversion from ambiguous grammar into an unambiguous grammar is not always possible. Some methods are used to remove ambiguity from CFG. Methods are given below.

- By fixing the grammar
- By adding the precedence rules
- By adding grouping rules
- By using semantics

These constraints are implemented using the following rules-

Rule-01:

The precedence constraint is implemented using the following rules-

- The level at which the production is present defines the priority of the operator contained in it.
- The higher the level of the production, the lower the priority of operator.
- The lower the level of the production, the higher the priority of operator.

Rule-02:

The associativity constraint is implemented using the following rules-

- If the operator is left associative, induce left recursion in its production.
- If the operator is right associative, induce right recursion in its production.

Problem-01:

Convert the following ambiguous grammar into unambiguous grammar-

E → E + E / E - E / E* E/id

To convert the given grammar into its corresponding unambiguous grammar, we implement the precedence and associativity constraints.

We have-

- Given grammar consists of the following operators- + , - , *

The priority order is- * > - > +

where-

- - operator is left associative
- + operator is left associative

Using the precedence and associativity rules, we write the corresponding unambiguous grammar as-

E → E + T / E-T/T

T → T * F / F

F → id

Unambiguous Grammar

➢ **Language Ambiguous**

- Language of grammar is the set of all strings that can be generated from that grammar.
- Finite language consists of finite number of strings.
- Infinite language consists of infinite number of strings.

If every grammar that generates language L is ambiguous, then language L is called as Inherently ambiguous language.

OR

A language is inherently ambiguous if all the grammars which generate that language are ambiguous.

- The term “inherently ambiguous language” is used instead of “ambiguous language”.
- This is because all the grammars which generate that language are ambiguous.

Important Points

- If a grammar is ambiguous, it does not imply that its language will be ambiguous too.
- If a grammar is ambiguous, its language may be unambiguous.
- If a grammar is ambiguous, its language will be unambiguous when there exists at least one unambiguous grammar which generates that language.

Example : Consider the following language L = a^n , n>=0

- This language consists of strings having any number of a's.
- Few grammars which generate this language are-

Grammar-01: S → aS / ∈ (Unambiguous Grammar)
Grammar-02: S → aS / a / ∈ (Ambiguous Grammar)
Grammar-03: S → aS / Sa / ∈ (Ambiguous Grammar)
Grammar-04: S → aS / a / Sa / ∈ (Ambiguous Grammar)

- All the above grammars generate the same language L = a^n , n>=0.
- Grammar-01 is unambiguous.
- Since there exists at least one unambiguous grammar which generates language L.
- Therefore, L is not an inherently ambiguous language.

➢ PUSHDOWN AUTOMATA(PDA)

- Pushdown automata is a way to implement a CFG in the same way we design DFA for a regular grammar. A DFA can remember a finite amount of information, but a PDA can remember an infinite amount of information.
- Basically a pushdown automaton is − **"Finite state machine" + "a stack"**

A stack does two operations −

Push − a new symbol is added at the top.
Pop − the top symbol is read and removed.

- A PDA is more powerful than FA. Any language which can be acceptable by FA can also be acceptable by PDA. PDA also accepts a class of language which even cannot be accepted by FA. Thus PDA is much more superior to FA.
- Think of an ε-NFA with the additional power that it can manipulate a stack.
- Its moves are determined by: 1. The current state (of its "NFA"), 2. The current input symbol (or ε), and 3. The current symbol on top of its stack.

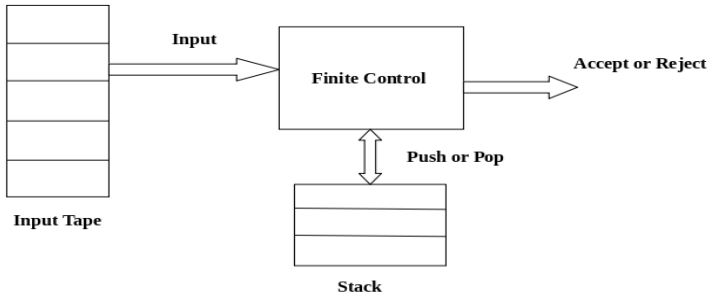


Fig: Pushdown Automata

PDA Components:

Input tape: The input tape is divided in many cells or symbols.
Read header : The input head is read-only and may only move from left to right, one symbol at a time.

Finite control: The finite control has some pointer which points the current symbol which is to be read.

Stack: The stack is a structure in which we can push and remove the items from one end only. It has an infinite size. In PDA, the stack is used to store the items temporarily.

Formal definition of PDA:

A PDA can be formally described as a **7-tuples** $(Q, \sum, \Gamma, \delta, q_0, Z_0, F)$ −

- **Q** is the finite number of states
- $\sum$ is input alphabet
- Γ(gamma) is stack symbols
- δ (Delta) is the transition function: $Q \times (\sum \cup \{\varepsilon\}) \times S \rightarrow Q \times \Gamma^*$
- $\mathbf{q_0}$ is the initial state ($q_0 \in Q$)
- Z_0 is the initial stack top symbol ($Z_0 \in S$)
- **F** is a set of accepting states ($F \subseteq Q$)

Stack Operations in PDA:

a) Push : $\delta(q,a,X)=(p,YX)$ replaces X with YX (pushes Y).
b) Pop : $\delta(q,a,X)=(p,\epsilon)$ replaces X with nothing (pops X).
c) No Operation (Skip) : $\delta(q,a,X)=(p,X)$

- **ϵ-**Transitions**:** $\delta(q,\epsilon,X)$ allows state changes or stack operations without reading input.

*Example***:**

δ(q0,a,Z0)=(q0,XZ0) In state $q0$, if input is 'a' and stack top is $Z0$, move to $q0$ and push X

$\delta(q1,\epsilon,X)=(q2,\epsilon)$: Pop X without consuming input

➢ **Pushdown Automata types**

Pushdown automata come in two main flavours:

1. Deterministic pushdown automaton (DPDA)
2. Nondeterministic pushdown automaton (NPDA).

The key difference lies in how they handle transitions from one state to another.

1. Deterministic Pushdown Automaton (DPDA)

In a DPDA, for each combination of current state, input symbol, and stack top symbol, there is at most one valid transition.

$\delta : Q\times(\Sigma\cup\{\epsilon\})\times\Gamma \rightarrow (Q\times\Gamma^*)$

2. Nondeterministic Pushdown Automaton (NPDA)

In an NPDA, there can be multiple possible transitions for a given combination of current state, input symbol, and stack top symbol.

Non-determinism allows for more flexibility in the computation, as the automaton can explore multiple paths simultaneously.

$\delta : Q\times(\Sigma\cup\{\epsilon\})\times\Gamma \rightarrow 2^{(Q\times\Gamma^*)}$

- DPDA is a restricted form of NPDA; every DPDA can be converted to NPDA, but not every NPDA can be converted to DPDA.
- NPDA can recognize all Context-Free Languages (CFLs)
- DPDA recognizes only Deterministic CFLs (DCFLs)
- DCFL is subset of CFL

➢ **Representation of Pushdown automata:**

The pushdown automaton is represented by the following types;

1. Transaction function of pushdown automata:
2. Graphical Notation of pushdown automata (PDA) / Transition Diagram

1. Transaction functions of pushdown automata:

The transaction function of pushdown automata has the following form;

$$\delta: (Q \times (\Sigma \cup \{\varepsilon\}) \times X) \rightarrow (p, \gamma)$$

δ is now a function of three arguments.

The first two arguments are the same as before:

(i) The state

(ii) The symbol of input alphabet or 'ε' .

(iii) The third argument is the symbol on top of the stack. Just as the input symbol is 'consumed' when the function is applied, the stack symbol is also "consumed" (removed from the stack).

γ → the current stack contents, with the top of the stack on the left

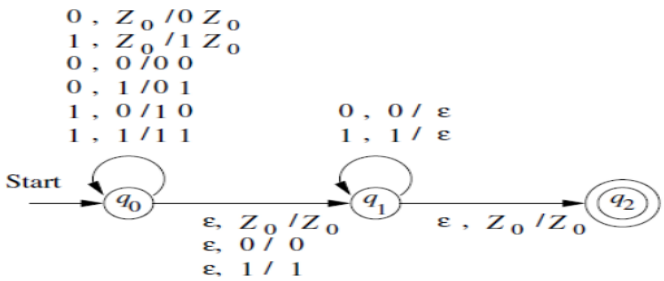
2. Graphical Notation of pushdown automata (PDA) / Transition Diagram :

The graphical representation of the PDA's consists;

1. The node corresponds to the states of the PDA.
2. An arrow labelled Start indicates the start state, and doubly circled states are final or accepting as for finite automata.
3. The arcs correspond to transition of the PDA in the following sense.
   input symbol, stack top / new stack top

- Consider a CFL $L_{ww^r} = \{ ww^r : w \in \{0,1\}^* \}$
- A pushdown automaton P for the language L_{ww^r} is as follows:

$$P = (\{q_0,q_1,q_2\}, \{0,1\}, \{0,1,Z_0\}, \delta, q_0, Z_0, \{q_2\})$$



➢ **Instantaneous description**

The Instantaneous description is called as an informal notation, and explains how a Push down automata (PDA) computes the given input string and makes a decision that the given string is accepted or rejected.

A ID is a triple (q, w, α), where:

- q is the current state.
  - w is the remaining input alphabet.
  - α is the current contents of the PDA stack.

- Instantaneous Description = current state + remaining input + current stack
- Generally, the leftmost symbol indicates the top of the stack α and the bottom at the right end. This type of triple notation is called an instantaneous description or ID of the pushdown automata.
- In short an instantaneous Description (ID) or configuration of a pushdown automaton (PDA) describes its execution status at any time.
- A move from one instantaneous description to another will be denoted by the symbol '⊢' thus
  $(q_0, aw, z_0) \vdash (q_1, w, yz_0)$

⊢ sign is called a "turnstile notation" and represents one move.

⊢* sign represents a sequence of moves.

For example, (p, b, T) ⊢ (q, w, α)

This implies that while taking a transition from state p to state q, the input symbol 'b' is consumed, and the top of the stack 'T' is replaced by a new string 'α'

➢ Pushdown Automata Acceptance - Types

There are two different ways to define PDA acceptability.

a) Final State Acceptability

In final state acceptability, a PDA accepts a string when, after reading the entire string, the PDA is in a final state. From the starting state, we can make moves that end up in a final state with any stack values. The stack values are irrelevant as long as we end up in a final state i.e. stack may or may not be empty

Language of PDA acceptance by Final state

For a PDA (Q, ∑, Γ, δ, q0 , Z, F), the language accepted by the set of final states F is −

$L(PDA) = \{w \mid (q0, w, z) \vdash^* (q, \varepsilon, x), \ q \in F\}$

for any input stack string x.

b) Empty Stack Acceptability

Here a PDA accepts a string when, after reading the entire string, the PDA has emptied its stack.

Let P = (Q, Σ, Γ, δ, q0 , Z0) be a PDA.

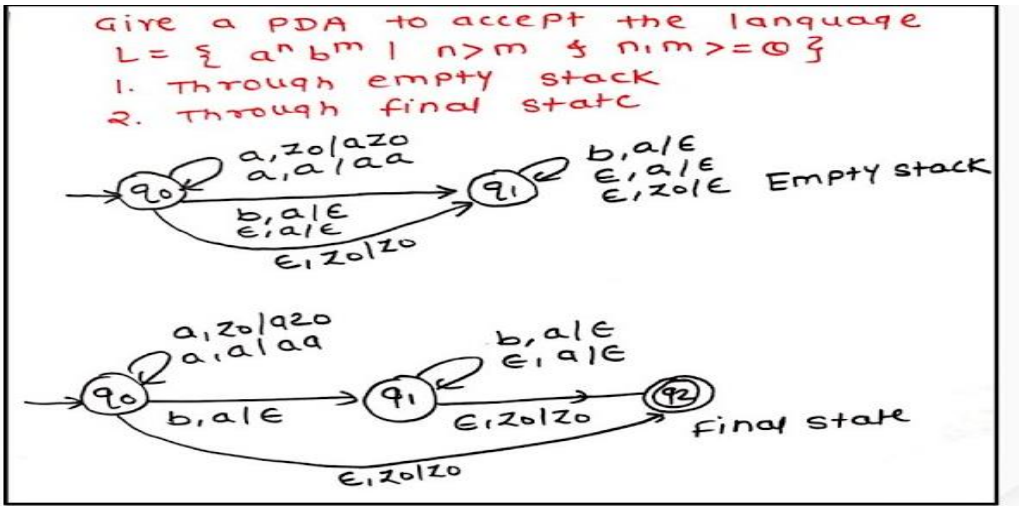
The language accepted by the empty stack is −

$L(PDA) = \{w \mid (q0, w, z) \vdash^* (q, \varepsilon, \varepsilon), q \in Q\}$

Since final states are irrelevant for PDAs that accept by empty stack, we do not define final states for those PDAs. – That is, a PDA that accepts by empty stack is 6 tuple (not 7 tuple).

Equivalence of Acceptance by Final State and Empty Stack

- The language accepted by empty stack PDA will also be accepted by final state PDA.
- Language accepted by final state PDA is also acceptable by empty stack PDA.



Example-1 :

Design a PDA for accepting a language $\{a^n b^{2n} \mid n>=1\}$.

In this language, n number of a's should be followed by $2n$ number of b's. Hence, we will apply a very simple logic, and that is if we read single 'a', we will push two a's onto the stack. As soon as we read 'b' then for every single 'b' only one 'a' should get popped from the stack.

Thus, the PDA, M = {Q, ∑, Γ, δ, q0, Z, F}, where Q = {q0, q1, q2} and Σ = { a, b },

Γ = {a, Z}, F={q2} and δ is given by the following ID:

δ(q0, a, Z) = (q0, aaZ)

δ(q0, a, a) = (q0, aaa)

δ(q0, b, a) = (q1, ε)

δ(q1, b, a) = (q1, ε)

δ(q1, ε, Z) = (q2, ε)

Instantaneous Descriptions

• Now we will simulate this PDA for the input string "aaabbbbbb".

δ(q0, aaabbbbbb, Z) ⊢ δ(q0, aabbbbbb, aaZ)
⊢ δ(q0, abbbbbb, aaaaZ)
⊢ δ(q0, bbbbbb, aaaaaaZ)
⊢ δ(q1, bbbbb, aaaaaZ)
⊢ δ(q1, bbbb, aaaaZ)
⊢ δ(q1, bbb, aaaZ)
⊢ δ(q1, bb, aaZ)
⊢ δ(q1, b, aZ)
⊢ δ(q1, ε, Z)
⊢ δ(q2, ε)
ACCEPT

Example Define the pushdown automata for language $\{a^n b^n \mid n > 0\}$

Solution : By Empty stack

Say, M = {Q, ∑, Γ, δ, q0 , Z, F}, where Q = { q0, q1 } and Σ = { a, b } and Γ = { A, Z } and δ is given by :

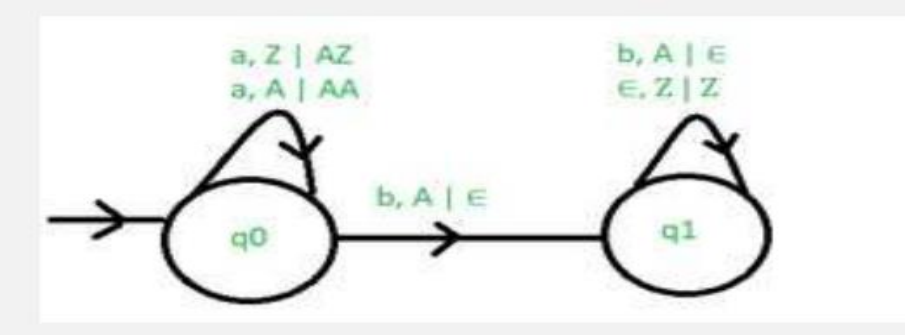
δ(q0, a, Z) = { (q0, AZ) }

δ(q0, a, A) = { (q0, AA) }

δ(q0, b, A) = { (q1, ∈) }

δ(q1, b, A) = { (q1, ∈) }

δ(q1, ∈, Z) = { (q1, ∈) }



EQUIVALENCE OF PUSHDOWN AUTOMATA AND CFG

CFG and PDA are equivalent in power: a CFG generates a context-free language and a PDA recognizes a context-free language.

If a grammar G is context-free, we can build an equivalent nondeterministic PDA which accepts the language that is produced by the context-free grammar G. A parser can be built for the grammar G.

Also, if P is a pushdown automaton, an equivalent context-free grammar G can be constructed where

L(G) = L(P)

- **Conversion from CFG to PDA:**

  PDA = (Q, Σ, Γ, δ, q0, z0)

  - States: just one state Q = {q}
  - Stack symbols: Γ = V∪Σ (all variables + terminals)
  - Start stack symbol: S (start symbol of CFG)
  - Acceptance: by empty stack
  - Transitions: two types:

*The first symbol on R.H.S. production must be a terminal symbol*GNF form). The following steps are used to obtain PDA from CFG accepting by empty stack is:

Step 1: Convert the given productions of CFG into GNF.

Step 2: The PDA will only have one state {q}.

Step 3: For non-terminal symbol, add the following transition function/rule:
If the production rule is $A \rightarrow \alpha$
Then the transition function/rule will be: $\delta(q, \varepsilon, A) = (q, \alpha)$
Ex: if $A \rightarrow aSA$ is the production rule then the transition function will be $\delta(q, \varepsilon, A) = (q, aSA)$

Step 4: Now, for each terminal symbol, add the following transition function/rule:
If 'a' is the terminal symbol
Then the transition function/rule will be: $\delta(q, a, a) = (q, \varepsilon)$

Example: Convert the following context free grammar to a PDA that accepts the
same language.

S → 0SX | 1SY | ε

X → 1

Y → 0

Solution:

Step 1: The given productions of CFG are in GNF already.

Step 2: Let 'q' be the PDA state.

Step 3: For non-terminal symbol, add the following transition function/rule:

1. δ(q, ε, S) = (q, 0SX) this for S → 0SX
2. δ(q, ε, S) = (q, 1SY) this for S → 1SY
3. δ(q, ε, S) = (q, ε) this for S → ε
4. δ(q, ε, X) = (q, 1) this for X → 1
5. δ(q, ε, Y) = (q, 0) this for Y → 0

Step 4: Now, for each terminal symbol, add the following transition function/rule:

1. δ(q, 0, 0) = (q, ε) this for terminal '0'
2. δ(q, 1, 1) = (q, ε) this for terminal '1'

So, finally the PDA can be

M = {{q}, {0, 1}, δ , {S, X, Y, 0, 1}, q , S, q}

The transition rules δ can be:

1. δ(q, ε, S) = (q, 0SX)
2. δ(q, ε, S) = (q, 1SY)
3. δ(q, ε, S) = (q, ε)
4. δ(q, ε, X) = (q, 1)
5. δ (q, ε, Y) = (q, 0)
6. δ(q, 0, 0) = (q, ε)
7. δ(q, 1, 1) = (q, ε)

- **Conversion from PDA to CFG**

  To convert a PDA into an equivalent CFG, we assume the PDA M = {Q, ∑, Γ, δ, q0 , Z, F}, accepts a language L by an empty stack.

  The CFG G = (V, T , P, S) is constructed following specific rules that ensure it generates the same language as the PDA.

  i) *Terminals* in CFG = Input symbols in PDA

  ii) *Variables* in CFG from PDA

  Start symbol 'S' and

  [p x q] where p, q are states in Q, x is stack symbol in Γ

  iii) *Productions* (P)

- Start Symbol Production

  For every state q_i in the PDA, we add a production rule to the CFG:

  S→[q0,z0,qi]

  Where q0 is initial state of PDA, Z0 is initial stack symbol

If the PDA has **n** states, there will be **n** productions originating from the start symbol S.

- For every pop operation

Let δ (q, a, x) = (p, ε) then production rule will be [q x p] → a

Let δ (q,ε, x) = (p, ε) then production rule will be [q x p] →ε

- For every push or skip operation

Let δ (q, a, x) =(p, $y_1 y_2 y_3 \ldots y_k$) then production rules will be

[p x r_k] → a [p y_1 r_1] [r_1 y_2 r_2] [r_2 y_3 r_3] …… [r_{k-1} y_k r_k] for all state r_1 ,$r_{2, \ldots}r_k$

Example

The PDA **is** M = ({q0,q1}, {0,1}, {X, Zo}, D, q0, Zo, {}) with δ (delta):

δ (q0,0,Z0) = (q0,XZo)

δ (q0,0,X) = (q0,XX)

δ (q0,1,X) = (q1,ε)

δ (q1,1,X) = (q1,ε)

δ (q1,e,X) = (q1,ε)

δ (q1,e,Zo) = (q1,ε)

The CFG

Obviously, T = {0,1} (same as sigma from the PDA)

We will call the Start Symbol "S".

V = (S, [q0,X,q0], [q0,X,q1], [q1,X,q0], [q1,X,q1],
[q0,Zo,q0], [q0,Zo,q1], [q1,Zo,q0], [q1,Zo,q1])

The Productions

- First, generate the productions from S:
  S → [q0,Zo,q0]
  S → [q0,Zo,q1]

The productions for δ (q0,0,Z0) = (q0,XZo)

[q0,Zo,q0] → 0[q0,X,q0] [q0,Zo,q0]
[q0,Zo,q0] → 0[q0,X,q1] [q1,Zo,q0]
[q0,Zo,q1] → 0[q0,X,q0] [q0,Zo,q1]
[q0,Zo,q1] → 0[q0,X,q1] [q1,Zo,q1]

The productions for δ (q0,0,X) = (q0,XX)

[q0,X,q0] → 0[q0,X,q0] [q0,X,q0]
[q0,X,q0] → 0[q0,X,q1] [q1,X,q0]
[q0,X,q1] → 0[q0,X,q0] [q0,X,q1]
[q0,X,q1] → 0[q0,X,q1] [q1,X,q1]

The productions for δ (q0,1,X) = (q1,ε) [q0,X,q1] → 1

The productions forδ (q1,1,X) = (q1,ε) [q1,X,q1] → 1

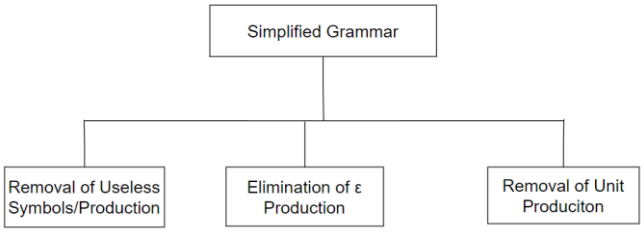
The productions forδ (q1,ε,X) = (q1,ε) [q1,X,q1] → ε

The productions forδ (q1,ε,Zo) = (q1,ε) [q1, Zo,q1] → ε

UNIT – 4

Simplification of Context-Free Grammars

Simplification of Context-Free Grammars (CFGs) refers to the process of reducing grammatical rules and symbols to their minimal necessary form while preserving the language generated by the grammar. This involves removing redundant or unnecessary symbols, rules, and productions without altering the language accepted by the grammar.



- **Useless Symbols**

If a symbol does not exist on the right-hand side of the production rule and does not participate in the derivation of any string, that symbol is considered a useless symbol. Similarly, if a variable does not participate in the derivation of any string, it is useless.
There are 2 types

a) Non-generating symbols --- symbols that cannot be used to derive any terminal string

b) Non-reachable symbols --- symbols that cannot be reached from start symbol of Grammar

Ex: S→ AB, A→ a, B → b C→ c where C is non-reachable

Ex : S→ AB , A → a, B→ bB where B is non generating

Steps to remove useless symbols

a) Find non-generating variables and delete them, along with all productions involving non-generating symbols

b) Find non-reachable variables in the resulting grammar, and delete them along with all productions involving non-reachable variables

Example:

Remove all useless productions from the following grammar:

$$E \longrightarrow aEb \mid bEa \mid EE \mid ab \mid a \mid B$$
$$A \longrightarrow a$$

Solution:

• In this grammar, we can see that from the starting symbol E, we can reach B, but from B the grammar is not able to generate the string. So, B is a useless symbol.

• Similarly, the non-terminal A is generating the string ‘a' but we are not able to reach A from the starting symbol E. Hence, in the given grammar, B is a useless symbol, while A→a is a useless production.

• Therefore, both are simply removed from the grammar, and after removal the grammar

E → aEb | bEa | EE |ab|a

Null Productions

• Production of the form **A→ε** are called empty or null productions. Thus, we can define null productions as any production in the grammar which generates ε directly (in a single step) or indirectly (in many steps).

• It is also called ε (or empty) production and the non-terminals which lead to ε (empty) are called nullable non-terminals or variables.

• If ε is an element of the language generated by the grammar then we cannot remove ε; otherwise we can remove ε productions.

Steps to Remove Null Productions

Step 1: Identify Null Productions

Check the grammar for productions of the form A → ε. Identify any non-terminal symbols that can derive ε through a series of productions.

Step 2: Find Productions Containing Nullable Variables

For each identified null production (A → ε), locate all other productions where 'A' appears on the right-hand side.

Step 3: Replace Nullable Variables with Epsilon

For every production we get in Step 2, create new productions by replacing each occurrence of the nullable variable ('A' in our example) with ε. Add these newly generated productions to the grammar.

Example: Remove the null productions from the following grammar

S → ABAC
A → aA / ϵ
B → bB / ϵ
C → c

Solution:

We have two null productions in the grammar A → ϵ and B → ϵ. To eliminate A → ϵ we have to change the productions containing A in the right side.

Those productions are S → ABAC and A → aA.

So replacing each occurrence of *A by ϵ*, we get four new productions

S → ABC / BAC / BC
A → a

Add these productions to the grammar and eliminate A → ϵ.

S → ABAC / ABC / BAC / BC
A → aA / a
B → bB / ϵ
C → c

To eliminate B → ϵ we have to change the productions containing B on the right side. Doing that we generate these new productions:

S → AAC / AC / C
B → b

Add these productions to the grammar and remove the production B → ϵ from the grammar. The new grammar after removal of ϵ – productions is:

S → ABAC / ABC / BAC / BC / AAC / AC / C
A → aA / a
B → bB / b
C → c

- **Unit Production**

In context-free grammars, a unit production is a production rule of the form "A → B", where both 'A' and 'B' represent non-terminal symbols.
By removing them, we can achieve a more concise and efficient grammar without affecting the language it generates.
Step 1: To remove A → B, add production A → x to the grammar rule whenever B → x occurs in the grammar.
Step 2: Delete A → B from the grammar.
Step 3: Repeat the first and the second steps until all the unit productions are removed.

Example: Remove the unit productions from the following grammar
S → AB
A → a
B → C / b
C → D
D → E
E →a
Solution:
There are 3 unit production in the grammar
B → C
C → D
D → E
For production D → E there is E → a so we add D → a to the grammar and add D → E from the grammar. Now we have C → D so we add a production C → a tp the grammar and delete C → D from the grammar. Similarly we have B → C by adding B → a and removing B → C we get the final grammar free of unit production as:

S → AB	A → a
B → a / b	C → a
D → a	E →a

We can see that C, D and E are unreachable symbols so to get a completely reduced grammar we remove them from the CFG. The final CFG is :
S -> AB
A -> a
B -> a / b

Question: Identify and remove the unit productions from the following CFG
S → S + T/ T
T → T * F/ F
F → (S)/a

Solution :
S → T and T → F are the two unit productions in the CFG.
For productions T → F we have F → (S)/a so we add T → (S)/a to the grammar and remove T→ F from the grammar. Now for production S → T we have production T → T * F/(S)/a so we add
S → T * F/(S)/a to the grammar. So the grammar after removal of unit production is:
S→S + T/ T * F/ (S)/ a
T -> T * F/ F
F -> (S)/ a

➢ Normal forms of CFG

Two commonly used normal forms are

1. Chomsky normal Form
2. Greibach normal Form

1. Chomsky Normal Form

CFG (Context-Free Grammar) is a grammar model used to generate all possible patterns of strings in programming languages.
In Chomsky Normal Form, we have restrictions on the length of RHS of the grammar rules i.e, RHS should be **two non-terminal symbols** otherwise **a single terminal.**
These rules can be expressed in the form

- A non-terminal symbol generating two non-terminal symbols (A → BC)
- A non-terminal symbol generating a terminal symbol (A → a)
- A start symbol generating ε (S → ε)

2. **Greibach Normal Form (GNF)**:

A CFG is in GNF if all its production rules are of the form:

- A non-terminal that generates a terminal followed by any number of non-terminals.

  A → aα, where A is a non-terminal symbol, a is a terminal symbol, and α is a string of zero or
  more non-terminal symbols.
- A non-terminal that generates a terminal i.e. A→ a
- A start symbol that generates ε. S→ €

➢ Steps for Converting CFG to CNF

Follow the below steps for converting context-free grammar to Chomsky Normal Form.

- If start symbol S is at the RHS of any production in the grammar, create a new production as: S0→S where S0 is the new start symbol.
- Remove useless productions, null productions and unit productions from the grammar.
- Eliminate terminals from RHS if they are the case with other terminals or non-terminals (e.g., the production rule A → aB breaks down as A → CB and C → a)
- Eliminate RHS with more than two non-terminals (e.g., the production rule A → ABC collapses into A → DC and D → AB)

Example:

Convert the following CFG into CNF.

A → 01XY, X → 1XY | ε, Y → YX0 | X | ε

Solution:

• First, eliminate all null productions, unit productions, and useless productions.

• Step 1: Remove null productions. X and Y are nullable non-terminals. Removal of null productions will give us the following grammar.

$A \longrightarrow 01XY \mid 01X \mid 01Y \mid 01$

$X \longrightarrow 1XY \mid 1X \mid 1Y \mid 1$

$Y \longrightarrow YX0 \mid X \mid Y0 \mid X0 \mid 0$

Step 2: Remove unit productions. Here, only one unit production Y → X is present.

So, We may simply add the production

Y → 1XY | IX | 1Y | 1

and delete Y → X

• Step 3: Apply some new non-terminals, if required, to the reduced grammar and restrict the productions to be in CNF

$A \longrightarrow 01XY \mid 01X \mid 01Y \mid 01$

$X \longrightarrow 1XY \mid 1X \mid 1Y \mid 1$

$Y \longrightarrow YX0 \mid 1XY \mid 1X \mid 1Y \mid 1 \mid Y0 \mid X0 \mid 0$

Let us assume $S \longrightarrow 0$ and $T \longrightarrow 1$ are the productions; now, the above productions will become

$A \longrightarrow STXY \mid STX \mid STY \mid ST$

$X \longrightarrow TXY \mid TX \mid TY \mid 1$

$Y \longrightarrow YXS \mid TXY \mid TX \mid TY \mid 1 \mid YS \mid XS \mid 0$

$S \longrightarrow 0$ and $T \longrightarrow 1$

Here, some of the productions remain unchanged because they are already in CNF. Now, we have many productions and their RHS is too long. So, let us assume the following substitutions:

$R_1 \longrightarrow XY$

$R_2 \longrightarrow ST$

$R_3 \longrightarrow YX$

After substituting these productions we obtain the final CFG which is in CNF as follows

$A \longrightarrow R_2R_1 \mid R_2X \mid R_2Y \mid ST$

$X \longrightarrow TR_1 \mid TX \mid TY \mid 1$

$Y \longrightarrow R_3S \mid TR_1 \mid TX \mid TY \mid 1 \mid YS \mid XS \mid 0$

$R_1 \longrightarrow XY$

$R_2 \longrightarrow ST$

$R_3 \longrightarrow YX$

$S \longrightarrow 0$ and $T \longrightarrow 1$

- **Left Recursion**

A grammar is left recursive if and only if there exists a non-terminal symbol that can derive to a sentential form with itself as the leftmost symbol

Left Recursion is a common problem that occurs in grammar during parsing in the syntax analysis part of compilation

It is important to remove left recursion from grammar because it can create an infinite loop, leading to errors and a significant decrease in performance.

Left recursion is eliminated by converting the grammar into right recursive grammar.

If we have left recursive pair of productions $A \rightarrow A\alpha \mid \beta$ where β does not begin with an A. Then we can eliminate left recursion by replacing the pair of productions with new productions as following

With Left Recursion	After Eliminating Left Recursion
(i) $A \rightarrow A\alpha \mid \beta$	$A \rightarrow \beta A'$ $A' \rightarrow \alpha A' \mid €$
(ii) $A \rightarrow A\alpha_1 \mid A\alpha_2 \mid A\alpha_3 \mid \beta$	$A \rightarrow \beta A'$ $A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \alpha_3 A' \mid €$
(iii) $A \rightarrow A\alpha \mid \beta_1 \mid \beta_2 \mid \beta_3$	$A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \beta_3 A'$ $A' \rightarrow \alpha A' \mid €$
(iv) $A \rightarrow A\alpha_1 \mid A\alpha_2 \mid A\alpha_3 \mid \beta_1 \mid \beta_2 \mid \beta_3$	$A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \beta_3 A'$ $A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \alpha_3 A' \mid €$

- **Steps for Converting CFG into GNF**

  Step 1 − Convert the grammar into CNF. If the given grammar is not in CNF, convert it into CNF by removing nullable productions, unit productions, and useless productions.

  Step 2 − If the grammar consists of left recursion, eliminate it.

  - Step 3 − In the grammar, convert the given production rule into GNF form. If any production rule in the grammar is not in GNF form, convert it. For that Substitute the production of the one variable in the another variable whenever necessary.

Example

S → AY | XX

X → x | SX

Y → y

A → x

As the given grammar is already in CNF, and no left recursion, we can skip the first and the second steps and directly go to the third step.

The production rule X → SX is not in GNF. Thus, substitute S → AY | XX in the production rule X → SX as:

S → AY | XX

X → x | AYX | XXX

Y → y

A → x

The production rule S → AY and Y → AYX is not in GNF. Thus, substitute A → x in the production rule S → AY and Y → AYX as:

S → xY | XX

X → x | xYX | XXX

Y → y

A → x

We will now remove left recursion (X → XXX), then get:

S → xY | XX

X → xZ | xYXZ

Z → XXZ | ε

Y → y

A → x

Removal of the null production Z → ε would get:

S → xY | XX

X → xZ | xYXZ | x | xYX

Z → XXZ | XX

Y → y

A → x

The production rule S → XX is not in GNF; thus, substitute X → xZ | xYXZ | x | xYX in production rule S → XX as:

S → xY | xZX | xYXZX | xX | xYXX

X → xZ | xYXZ | x | xYX

Z → XXZ

Z → xZX | xYXZX | xX | xYXX

Y → y

A → x

The production rule C → XXZ is not in GNF, so substitute X → xZ | xYXZ | x | xYX in production rule Z → XXZ as:

S → xY | xZX | xYXZX | xX | xYXX

X → xZ | xYXZ | x | xYX

Z → xZXZ | xYXZXZ | xXZ | xYXXZ

Z → xZX | xYXZX | xX | xYXX

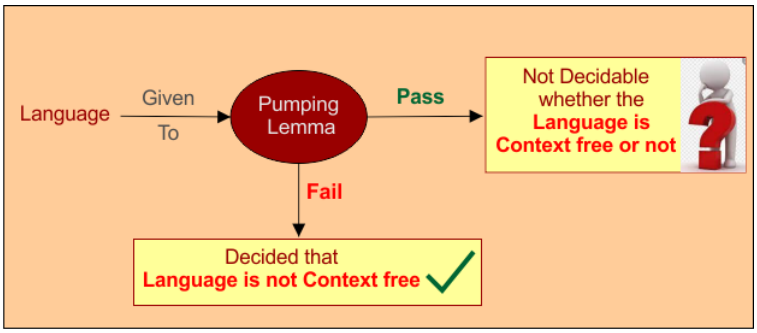
Y → y

A → x

Hence, this is the GNF form for the grammar G.

➢ Pumping Lemma for Context Free Languages

Pumping Lemma for CFL states that for any Context-Free Language L, it is possible to find two substrings that can be 'pumped' any number of times and still be in the same language. We break its strings into five parts for any language L and pump the second and fourth substrings. If any string does not satisfy its conditions, then the language is not CFL.



Pumping Lemma Test (PLT) - For Context Free Language

If L is a context-free language, there is a constant 'n' that depends exclusively on L, such that if w∈ L and |w| >= n, w can be divided into five pieces, w = uvxyz, meeting the following requirements.

- **|vxy| <=n**
- **|vy| not = ε**
- **For all k>=0, the string uv^kxy^yz∈L**

Steps to apply pumping lemma

- Assume that L is context-free.
- The pumping length will be n.
- All strings longer than the length(n) can be pumped i.e |w|>=n.
- Now we have to find a string 'w' in L such that |w|>=n.
- We will divide string 'w' into uvxyz.
- Now show that $uv^kxy^kz \notin L$ for some constant k.
- Then, we have to consider the ways that w can be divided into uvxyz.
- Show that none of these can satisfy all the 3 pumping conditions simultaneously.
- A string 'w' cannot be pumped (contradiction).

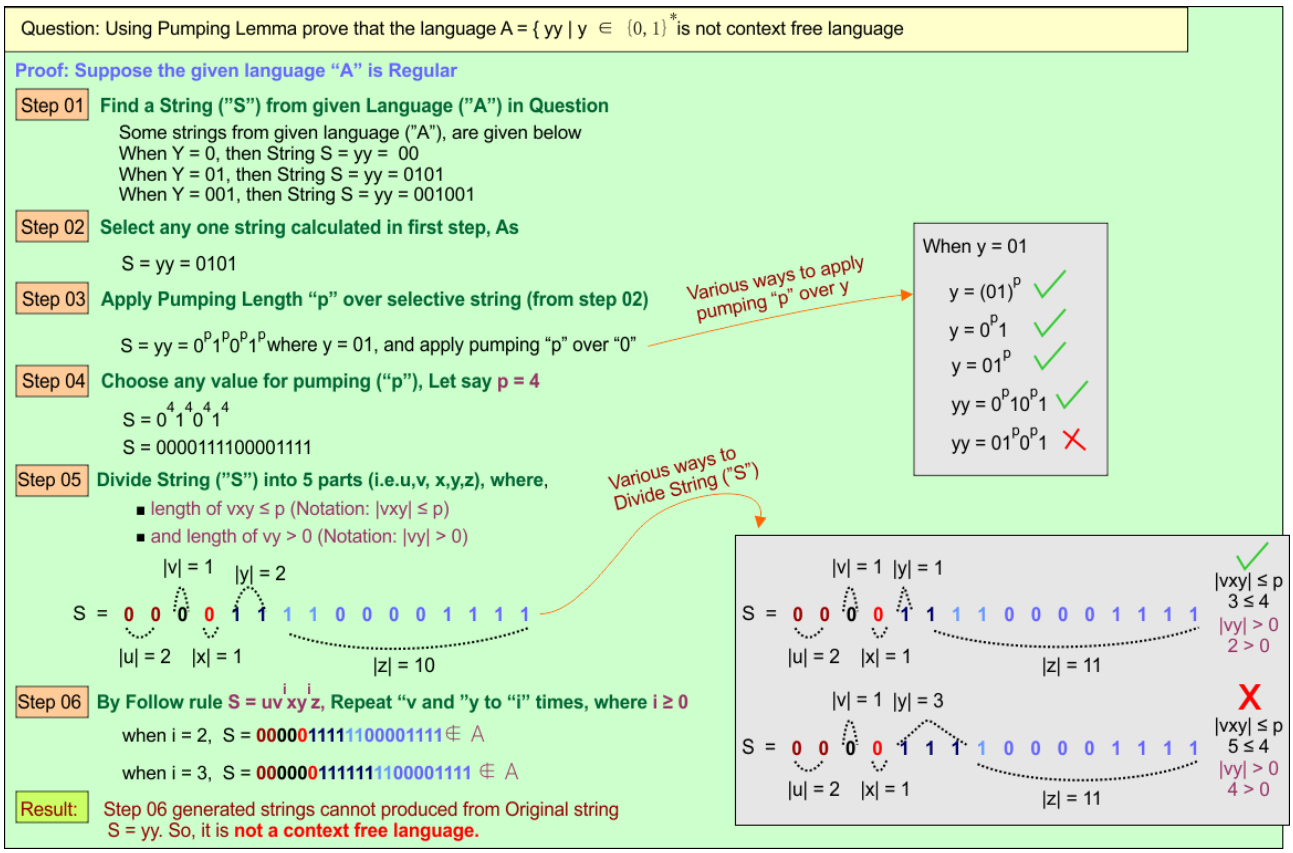
If L is context-free then L satisfies the pumping lemma.

If L satisfies the pumping lemma that does not mean L is context-free

If L does not satisfy the pumping lemma then L is not context-free.

If you show:

- Case 1 → not in L
- Case 2 → not in L
- But Case 3 → still in L
- So pumping lemma is satisfied.
- So you **CANNOT** conclude the language is not CFL.



➢ CLOSURE PROPERTIES OF CFL

1. Union

If L1 and L2 are context-free languages then L1 ∪ L2 is also context-free Language

Let L1 be language recognized by G1 = (V1, T1, P1, S1) and L2 the language recognized by G2 = (V2, T2, P2, S2). Assume that V1 ∩ V2 = ∅;

We need to show that L(G) = L(G1) ∪ L(G2) where S = S1 | S2

Example

If L1 = {$a^n b^n$ | n>0 } $S_1 \rightarrow aS_1b \mid ab$

L2 = {$c^m d^m$ | m> 0 } $S_2 \rightarrow aS_2d \mid cd$

L3(G) = L1(G) U L2(G) = $S \rightarrow S_1 \mid S_2$ $S_1 \rightarrow aS_1b \mid ab$

$S_2 \rightarrow aS_2d \mid cd$

Resultant language contains strings having equal no. of a's followed b's or equal no. of c's followed by d's

L3 = {$a^n b^n$ | $c^m d^m$ | m> 0, n>0 }

2. Concatenation

If L1 and L2 are context-free languages then L1.L2 is also context-free Language

Let L1 be language recognized by G1 = (V1, T1, P1, S1) and L2 the language recognized by G2 = (V2, T2, P2, S2). Assume that V1 ∩ V2 = ∅; $S \rightarrow S1.S2$

If L1 = {$a^n b^n$ | n>0 } $S_1 \rightarrow aS_1b \mid ab$

L2 = {$c^m d^m$ | n> 0 } $S_2 \rightarrow aS_2d \mid cd$

L3(G)= L1(G) .L2(G) = $S \rightarrow S_1.S_2$ $S_1 \rightarrow aS_1b \mid ab$

$S_2 \rightarrow aS_2d \mid cd$

Resultant language contains strings having equal no. of a's followed b's is concat with equal no. of c's followed by d's

$L3 = \{a^n b^n c^m d^m \mid m > 0, n > 0\}$

3. KleeneClosure

If **L** is a CFL, then its Kleene closure **L*** (zero or more repetitions of strings in **L**) is also a CFL.

If L1 is a CFL, then $L1^*$ is a CFL.

Let L1 be generated by the CFG, G1 = (V1, T1, P1, S1).

The grammar represented by L1 = L(G1)

Define the CFG, G, that generates $L1^*$ as follows: S= S. S1 | ε

Ex :

$L1 = \{a^n b^n \mid n > 0\}$ $L1(G) = S_1 \rightarrow aS_1b \mid ab$

$L1^*(G) = S \rightarrow S.\ S1 \mid \varepsilon S_1 \rightarrow aS_1b \mid ab$

4. Intersection

Context-free languages are not closed under intersection or complement.

Let $L1 = \{a^m b^m c^n : m, n \geq 0\}$ and let $L2 = \{a^m b^n c^n : m, n \geq 0\}$.

Then both L1 and L2 are context-free.

However, their intersection $L1 \cap L2$ is $\{a^n b^n c^n : n \geq 0\}$ which is not context-free.

5. Complement

The complement of a language L is the set of all strings that are not in L. In the context of a context-free grammar, the complement of a CFL is the set of all strings that are not generated by the CFG.

The complement of a Context-Free Language (CFL) is a Context-Sensitive Language (CSL). While CFLs are **not** closure under complementation, every CFL is also a CSL, and CSLs are closed under complementation.

$L1 = \{a^n b^n \mid n > 0\}$

$L1\ ' = \{w \in \{a,b\}^* \mid w \text{ is NOT of the form } a^n b^n\}$

6. Reversal

If L is a CFL, then its reversal L^R (where each string is reversed) is also a CFL.

Construction:

1. Take the context-free grammar G for L.
2. Modify each production so that the right-hand side of every rule is reversed.
3. The resulting grammar generates L^R, proving that CFLs are closed under reversal.

Example:

- *Original Grammar: S → 0S1 | 01*
- *Reversed Grammar: S → 1S0 | 10*

7. Homomorphism

A homomorphism is a function that replaces each symbol in a string with another string.
If L is a CFL and h is a homomorphism, then h(L) is also a CFL.
Example:

- *Let G have the production S → 0S1 | 01.*
- *Define a homomorphism h(0) = ab, h(1) = aa.*
- *Then, h(L(G)) will be generated by a new grammar with productions:*
  S → abSaa | abaa

This shows that CFLs are closed under homomorphism.

8. InverseHomomorphism

If L is a CFL and h is a homomorphism, then $h^{-1}(L)$ is also a CFL.
Construction:

- Instead of using a grammar, we use a PDA to construct the inverse homomorphism.
- The PDA for $h^{-1}(L)$ simulates the PDA for L but keeps track of the mapping of symbols.

Operation	Regular Languages (RL)	Context-Free Languages (CFL)
Union (∪)	✓ Closed	✓ Closed
Concatenation	✓ Closed	✓ Closed
Kleene Star (*)	✓ Closed	✓ Closed
Reversal	✓ Closed	✓ Closed
Intersection (RL ∩ RL / CFL ∩ CFL)	✓ Closed	✗ Not Closed
Complement	✓ Closed	✗ Not Closed
Difference	✓ Closed	✗ Not Closed
Homomorphism	✓ Closed	✓ Closed
Inverse Homomorphism	✓ Closed	✓ Closed

Decision properties of CFL

Problem	Regular Languages (RL)	Context-Free Languages (CFL)
Membership (Is w ∈ L)?)	✓ Decidable	✓ Decidable
Emptiness (Is L= ϕ)	✓ Decidable	✓ Decidable
Finiteness (Is L finite?)	✓ Decidable	✓ Decidable
Equivalence (L1 = L2 ?)	✓ Decidable	✗ Undecidable
Containment ((L1 ⊆ L2)?)	✓ Decidable	✗ Undecidable

➢ **Decision properties of CFL**

1. Algorithm To Decide Whether CFL Is Finite Or Not

For a given CFG, there exists an algorithm to decide whether its language is finite or not.

Step-01:

- Eliminating ∈ productions, Eliminating unit productions and Eliminating useless productions

Step-02:

- Draw a directed graph whose nodes are variables of the given grammar.
- There exists an edge from node A to node B if there exists a production of the form A → αBβ.

Now, following 2 cases are possible-

- Directed graph contains a cycle -- In this case, language of the given grammar is infinite.
- Directed graph does not contain any cycle -- In this case, language of the given grammar is finite.

Example

Check whether language of the following grammar is finite or not-

S → AB / a, A → BC / b, B → CC / c

Solution

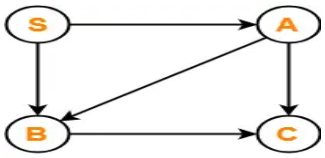
The given grammar is already completely reduced.

We will draw a directed graph whose nodes will be S , A , B , C.

Now,

- Due to the production S → AB, directed graph will have edges S → A and S → B.
- Due to the production A → BC, directed graph will have edges A → B and A → C.
- Due to the production B → CC, directed graph will have edge B → C.

The required directed graph is-



Directed Graph

Clearly,

- The directed graph does not contain any cycle.
- Therefore, language of the given grammar is finite.

Problem-02

Check whether language of the following grammar is finite or not-

S → XS / b , X → YZ, Y → ab, Z → XY

Solution

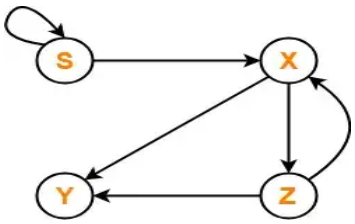
The given grammar is already completely reduced.

We will draw a directed graph whose nodes will be S , X , Y , Z.

Now,

- Due to the production S → XS / b, directed graph will have edges S → X and S → S.
- Due to the production X → YZ, directed graph will have edges X → Y and X → Z.
- Due to the production Z → XY, directed graph will have edges Z → X and Z → Y.

The required directed graph is-



Clearly,

- The directed graph contain cycles.
- Therefore, language of the given grammar is infinite.

2) Algorithm To Decide Whether CFL Is Empty Or Not-

If we cannot derive any string of terminals from the given grammar, then its language is called as an **Empty Language**. $L(G) = \phi$

For a given CFG, there exists an algorithm to decide whether its language is empty $L(G) = \phi$ or not.

- Remove all the useless symbols from the grammar.
- If the start symbol is found to be useless, then language is empty otherwise if start symbol can derive a terminal string then not empty L.

If $L(G) = \{ \in \}$ i.e. If the language generated by a grammar contains only a null string, then it is considered as non-empty.

Example-

Consider the following grammar-

S → XY, X → AX, X → AA, A → a, Y → BY, Y → BB, B → b

Now, let us check whether language generated by this grammar is empty or not.

The given grammar can be written as-

$S \rightarrow aabb, X \rightarrow aX, X \rightarrow aa, A \rightarrow a, Y \rightarrow bY, Y \rightarrow bb, B \rightarrow b$

Clearly,

- The start symbol generates at least one string (many more are possible).
- Therefore, start symbol is useful.
- Thus, language generated by the given grammar is non-empty.

3. Membership Problem

It is used to check whether a string is member of given language or not . CYK algorithm is used to do this.

CYK Algorithm

CYK Algorithm is a membership algorithm of context free grammar.

- It is used to decide whether a given string belongs to the language of grammar or not.
- It is also known as **CKY Algorithm** or **Cocke-Younger-Kasami Algorithm** after its inventors

CYK Algorithm operates only on Context Free Grammars given in Chomsky Normal Form.

To decide the membership of any given string, we construct a triangular table where-

- Each row of the table corresponds to one particular length of sub strings.
- The last cell of each row corresponds to strings of length 1.
- Second last cell of each row from right side corresponds to strings of length 2.
- 3rd cell of each row from right side corresponds to strings of length 3.
- The 1st cell in 1st row corresponds to string of length 'n'

	4	3	2	1
1	[1, 4]	[1, 3]	[1, 2]	[1, 1]
2	[2, 4]	[2 3]	[2, 2]	
3	[3, 4]	[3, 3]		
4	[4, 4]			

We will fill each box of X_{ij} with its $V_{i\,j}$.

$X_{i\,j}$ represents a sub string of "x" starting from location 'i' to location 'j'.

Example-
Consider $x = abcd$ is a string, then-
Number of sub strings possible = n(n+1)/2 = 4 x (4+1) / 2 = 10
We have-

x_{11} = a x_{22} = b x_{33} = c x_{44} = d x_{12} = ab x_{23} = bc
x_{32} = cd x_{13} = abc x_{24} = bcd x_{14} = abcd

V_{ij} represents a set of variables in the grammar which can derive the sub string **X_{ij}**.
If the set of variables consists of the start symbol, then it becomes sure-

- Sub string $X_{i\,j}$ can be derived from the given grammar.
- Sub string $X_{i\,j}$ is a member of the language of the given grammar.

Problem-

For the given grammar, check the acceptance of string w = baaba using CYK Algorithm-

S → AB / BC A → BA / a B → CC / b C → AB / a

Solution-

First, let us draw the triangular table.

- The given string is x = baaba. Length of given string = |x| = 5.
- So, Number of sub strings possible = (5 x 6) / 2 = 15.

	5	4	3	2	1
1	[1, 5]	[1, 4]	[1, 3]	[1, 2]	[1, 1]
2	[2, 5]	[2, 4]	[2 3]	[2, 2]	
3	[3, 5]	[3, 4]	[3, 3]		
4	[4, 5]	[4, 4]			
5	[5, 5]				

First fill the last cells in each row then 2Nd last cells in each row and so on

For 1st diagonal

Finding V_{11}

V_{11} represents the set of variables deriving x_{11}. x_{11} = b.

Only variable B derives string "b" in the given grammar. Thus, V_{11} = { B }

Finding V_{22}

V_{22} represents the set of variables deriving x_{22}. x_{22} = a.

Variables A and C derive string "a" in the given grammar. Thus, V_{22} = { A , C }

Finding V_{33}

- V_{33} represents the set of variables deriving x_{33}. x_{33} = a.
- Variables A and C derive string "b" in the given grammar. Thus, V_{33} = { A , C }

Finding V_{44}

V_{44} represents the set of variables deriving x_{44}. x_{44} = b.

- Only variable B derives string "b" in the given grammar. Thus, V_{44} = { B }

Finding V_{55}

V_{55} represents the set of variables deriving x_{55}. x_{55} = a.

- Variables A and C derives string "a" in the given grammar. Thus, V_{55} = { A , C }

For 2nd diagonal

Finding V_{12}

$V_{12} = V_{11} . V_{22}$

V_{12} = { B } { A , C }

V_{12} = { BA , BC }

∴ V_{12} = { A , S }

Finding V_{23}

$V_{23} = V_{22} . V_{33}$

V_{23} = { A , C } { A , C }

V_{23} = { AA , AC , CA , CC }

Since AA , AC and CA do not exist, so we have-

V_{23} = { CC }

∴ V_{23} = { B }

Finding V_{34}

$V_{34} = V_{33} . V_{44}$

V_{34} = { A , C } { B }

V_{34} = { AB , CB }

Since CB does not exist, so we have-

V_{34} = { AB }

∴ V_{32} = { S , C }

Finding V_{45}

$V_{45} = V_{44} . V_{55}$

V_{45} = { B } { A , C }

V_{45} = { BA , BC }

∴ V_{45} = { A , S }

For 3 rd diagonal

Finding V_{13}

$V_{13} = V_{11}. V_{23} \cup V_{12}. V_{33}$

$V_{13} = \{ B \} \{ B \} \cup \{ A , S \} \{ A , C \}$

$V_{13} = \{ BB \} \cup \{ AA , AC , SA , SC \}$

Since BB , AA , AC , SA and SC do not exist, so we have-

$V_{13} = \phi \cup \phi$

$\therefore V_{13} = \phi$

Finding V_{24}

$V_{24} = V_{22}. V_{34} \cup V_{23}. V_{44}$

$V_{24} = \{ A , C \} \{ S , C \} \cup \{ B \} \{ B \}$

$V_{24} = \{ AS , AC , CS , CC \} \cup \{ BB \}$

Since AS , AC , CS and BB do not exist, so we have-

$V_{24} = \{ CC \}$

$\therefore V_{24} = B$

Finding V_{35}

$V_{35} = V_{33}. V_{45} \cup V_{34}. V_{55}$

$V_{35} = \{ A , C \} \{ A , S \} \cup \{ S , C \} \{ A , C \}$

$V_{35} = \{ AA , AS , CA , CS \} \cup \{ SA , SC , CA , CC \}$

Since AA , AS , CA , CS , SA , SC and CA do not exist, so we have-

$V_{35} = \phi \cup \{ CC \}$

$V_{35} = \phi \cup \{ B \}$

$\therefore V_{35} = \{ B \}$

For 4 th diagonal

Finding V_{14}

$V_{14} = V_{11}. V_{24} \cup V_{12}. V_{34} \cup V_{13}. V_{44}$

$V_{14} = \{ B \} \{ B \} \cup \{ A , S \} \{ S , C \} \cup \{ \phi , B \}$

$V_{14} = \{ BB \} \cup \{ AS , AC , SS , SC \} \cup \{ B \}$

Since BB , AS , AC , SS , SC and B do not exist, so we have-

$V_{14} = \phi \cup \phi \cup \phi$

$\therefore V_{14} = \phi$

Finding V_{25}

$V_{25} = V_{22}. V_{35} \cup V_{23}. V_{45} \cup V_{24}. V_{55}$

$V_{25} = \{ A , C \} \{ B \} \cup \{ B \} \{ A , S \} \cup \{ B \} \{ A , C \}$

$V_{25} = \{ AB , CB \} \cup \{ BA , BS \} \cup \{ BA , BC \}$

Since CB does not exist, so we have-

$V_{25} = \{ AB \} \cup \{ BA , BS \} \cup \{ BA , BC \}$

$V_{25} = \{ S , C \} \cup \{ A \} \cup \{ A , S \}$

$\therefore V_{25} = \{ S , C , A \}$

For first cell

Finding V_{15}

$V_{15} = V_{11}. V_{25} \cup V_{12}. V_{35} \cup V_{13}. V_{45} \cup V_{14}. V_{55}$

V_{15} = {B} {S, C, A} ∪ { A, S} { B } ∪ { ϕ } { A,S } ∪ { ϕ } { A,C }
V_{15} = { BS , BC , BA } ∪ { AB , SB } ∪ { A , S } ∪ { A , C }
Since BS , SB , A , S and C do not exist, so we have-
V_{15} = { BC , BA } ∪ { AB } ∪ ϕ ∪ ϕ
V_{15} = { S , A } ∪ { S , C } ∪ ϕ ∪ ϕ
∴ V_{15} = { S , A , C }

Now,

- The value of V_{ij} is computed for each cell.
- We observe V_{15} contains the start symbol S.
- Thus, string x_{15} = baaba is a member of the language of given grammar.

After filling the triangular table, it looks like-

	5	**4**	**3**	**2**	**1**
1	{S,A,C}	**ø**	**ø**	{A,S}	{ B }
2	{S A,C}	{ B }	{ B }	{A,C}	
3	{ B }	[3, 4]	[3, 3]		
4	[4, 5]	{ B }			
5	{ A,C}				

Unit 3

1 Explain different types of grammar
2. Define formal definition of PDA. Give one example .
3. Explain how many ways PDA can be represented ?
4. Explain types of acceptance of PDA with example ?
5. Explain different types of derivation with example ?
6. Explain the PDA components ?
7. Write steps to convert CFG to PDA
8. Write steps to convert PDA to CFG

1. Define Grammar
2. Define CFG
3. Define Parse tree
4. What is sentential form of derivation
5. What are types of derivations of string
6. What are the types of PDAan
7. What is instantaneous description
8. What is the difference between FA and PDA
9. What is the relation between Regular language and Context free language
10. Define ambiguous grammar
11. Define ambiguous language
12. Give 3 examples of Regular language
13. Give 3 examples of Context free language
14. Give 3 examples of not Context free language

Unit -4

1. Explain the steps to simplify CFG
2. Explain steps to convert CFG to CNF with example
3. Explain steps to convert CFG to GNF with example
4. Explain Pumping lemma theorem for CFL
5. Explain closure properties of CFL
6. Explain Decision properties of CFL
7. Explain how to eliminate left recursion

Define unit production
Define null production
Define useless production
What are the types of normal forms of CFG
Define CNF
Define GNF
Define recursion grammar
Define Left recursion
What are the applications of pumping lemma